



ESTRUCTURA DE DATOS

Taller 02

II Semestre – 2026
ITI – ICCI



Docentes: Bastián Ruiz – Juan Nilo

Ayudantes: Martín Droguett – Daniel Durán

Fecha del enunciado: 21/09/2026 – 13:00

Fecha entrega taller: 25/10/2026 – 18:00

Extender el sistema desarrollado en el Taller 1 para clasificar automáticamente a los pacientes mediante un árbol de decisión y almacenarlos en árboles AVL según el servicio asignado. Se deberán aplicar árboles binarios, recorridos, búsqueda, balanceo y rotaciones AVL.

Contexto

El hospital mantiene los mismos servicios definidos en el Taller 1:

- Urgencias
- Medicina General
- Cardiología
- Neurología
- Traumatología
- Cirugía
- Pediatría
- Hospitalización

En esta versión, el servicio del paciente ya no viene indicado directamente en el archivo. El sistema deberá determinar el servicio correspondiente a partir de las características del paciente utilizando un árbol de decisión.

Flujo principal:

Archivo → Cola → Árbol de decisión → Servicio → AVL

Requerimientos

Carga de pacientes

El programa deberá leer un archivo de texto con la información de los pacientes. Cada paciente deberá contener, como mínimo:

```
ID;Nombre;Edad;Temperatura;PresionArterial;Dolor;TipoDolor;  
Trauma;SintomasNeurologicos;RequiereCirugia;DiasHospitalizacion
```

Los atributos deberán interpretarse de la siguiente forma:

- ID: identificador único del paciente.
- Edad: edad en años.
- Temperatura: temperatura corporal.
- PresionArterial: valor numérico definido por el grupo.
- Dolor: escala entre 0 y 10.
- TipoDolor: pecho, cabeza, abdomen, extremidad u otro.
- Trauma: 0 si no existe y 1 si existe.
- SintomasNeurologicos: 0 si no existen y 1 si existen.
- RequiereCirugia: 0 si no requiere y 1 si requiere.
- DiasHospitalizacion: cantidad estimada de días.

Los pacientes deberán incorporarse a una cola implementada manualmente, respetando el orden de llegada. No se podrá utilizar una estructura de la STL para reemplazar la cola solicitada.

Árbol de decisión

Cada paciente deberá ser evaluado antes de ser enviado a un servicio. El árbol de decisión deberá utilizar atributos del paciente para determinar el servicio correspondiente. Deberá permitir:

- Construcción del árbol.
- Recorridos.
- Evaluación de un paciente.
- Determinación del servicio resultante.

El criterio exacto de decisión deberá ser definido por el grupo y documentado en el README.

Ejemplo conceptual:

```
¿Edad < 14?
|-- Si: Pediatría
|
|-- No:
    ¿Trauma == 1?
    |-- Si:
        |   ¿RequiereCirugia == 1?
        |   |-- Si: Cirugia
        |   |-- No: Traumatología
        |
    |-- No:
        ¿SintomasNeurologicos == 1?
        |-- Si: Neurología
        |
        |-- No:
            ¿TipoDolor == "pecho"?
            |-- Si: Cardiología
            |
            |-- No:
                ¿Temperatura >= 38.5 o Dolor >= 8?
                |-- Si: Urgencias
                |
                |-- No:
                    ¿DiasHospitalizacion >= 2?
                    |-- Si: Hospitalización
                    |-- No: Medicina General
```

El ejemplo es ilustrativo. El árbol final deberá ser diseñado e implementado por el grupo y deberá cumplir las siguientes condiciones:

- Tener al menos una ruta válida hacia cada uno de los ocho servicios.
- Utilizar al menos cuatro atributos del paciente.
- Definir qué ocurre ante datos inválidos o insuficientes.
- Documentar la justificación de cada decisión.
- Clasificar correctamente casos de prueba representativos.

Árbol AVL por servicio

Una vez determinado el servicio, el paciente deberá almacenarse en el AVL correspondiente:

```
Hospital
|-- Urgencias -----> AVL
```

```
|-- Medicina General -> AVL
|-- Cardiologia -----> AVL
|-- Neurologia -----> AVL
|-- Traumatologia ----> AVL
|-- Cirugia -----> AVL
|-- Pediatria -----> AVL
|-- Hospitalizacion --> AVL
```

Cada AVL utilizará el ID como clave de ordenamiento. Se deberán implementar las rotaciones simples izquierda y derecha, además de las rotaciones dobles izquierda-derecha y derecha-izquierda.

Operaciones sobre los AVL

El sistema deberá permitir:

- Insertar un paciente.
- Buscar un paciente por ID o nombre.
- Mostrar recorrido inorden.
- Mostrar recorrido preorden.
- Mostrar recorrido postorden.
- Mostrar altura del árbol.
- Mostrar factor de balance.

Procesamiento

Los pacientes deberán ser procesados respetando el orden establecido por la Queue. Para cada paciente:

1. Se retira de la Queue.
2. Se evalúa mediante el árbol de decisión.
3. Se obtiene el servicio.
4. Se inserta en el AVL correspondiente.
5. Se informa el resultado.

Ejemplo de Ejecución

```
=== HOSPITAL MARMAJA ===
```

1. Atender pacientes
2. Ver departamentos
3. Buscar paciente
4. Salir

```
Seleccionar opcion: 1
```

```
=== ATENDIENDO PACIENTES ===
```

```
ID: 001
```

```
Nombre: Juan Perez
```

```
Edad: 25
```

```
Temperatura: 37.0
```

```
Dolor: 3
```

```
TipoDolor: otro
```

```
Trauma: 0
```

```
SintomasNeurologicos: 0
```

```
RequiereCirugia: 0
```

```
DiasHospitalizacion: 0
```

```
Ruta: Edad >= 14 -> Trauma == 0 ->
```

```
    SintomasNeurologicos == 0 -> TipoDolor != pecho ->
```

```
    Temperatura < 38.5 y Dolor < 8 -> DiasHospitalizacion < 2
```

```
Servicio asignado: Medicina General
```

```
Paciente insertado en el AVL de Medicina General.
```

```
----- Opción 2 -----
```

```
=== HOSPITAL MARMAJA ===
```

1. Atender pacientes
2. Ver departamentos
3. Buscar paciente
4. Salir

```
Seleccionar opcion: 2
```

```
=== ESTADO DE LOS DEPARTAMENTOS ===
```

```
Urgencias
```

```
Sin pacientes
```

```
Medicina General
```

```
Pacientes: Juan Perez (001)
```

Traumatologia
Sin pacientes

Cardiologia
Sin pacientes

Neurologia
Sin pacientes

Cirugia
Sin pacientes

Pediatría
Sin pacientes

Hospitalizacion
Sin pacientes

----- Opción 3 -----

=== HOSPITAL MARMAJA ===

1. Atender paciente
2. Ver departamento
3. Buscar paciente
4. Salir

Seleccionar opcion: 3

=== BUSCAR PACIENTE ===

Indique el ID o Nombre del paciente: 001

Buscando en AVL...

Paciente encontrado:

ID: 001

Nombre: Juan Perez

Edad: 25

Departamento: Medicina General

T: 36.8° | D: 3 | TD: otro | T: No | SN: No | RC: No | DH: Ninguno

Ruta de busqueda:

01

Consideraciones

- El taller deberá desarrollarse en C++.
- El taller puede realizarse de forma individual o grupos de hasta 3.
- Los árboles binarios y AVL deberán ser implementados manualmente.
- El ID será la clave de ordenamiento del AVL.
- Los servicios deberán mantenerse respecto al Taller 1.
- El árbol de decisión deberá ser diseñado por el grupo y documentado.
- Los casos de empate y valores inválidos deberán estar definidos.
- Se deberán controlar archivos inexistentes, datos inválidos y pacientes duplicados.
- Se deberán implementar destructores o mecanismos equivalentes para liberar correctamente la memoria.
- Se deberá liberar correctamente la memoria dinámica.
- El código deberá estar separado en archivos `.h` y `.cpp` cuando corresponda.
- Se deberá utilizar GitHub para el desarrollo del proyecto.
- La interfaz será de consola.
- Se evaluará el funcionamiento de las estructuras y no solamente el resultado final.

Obligatorio – Penalización en caso de incumplimiento:

- El README debe incluir integrantes (nombre, rut, nombre en Github, carrera, paralelo), instrucciones de compilación y ejecución.
- El repositorio debe mantener un historial de commits coherente durante todo el desarrollo.
- La implementación de las estructuras de datos (Queue, AVL, Árbol de Decisión), debe realizarse de forma manual.
- El taller debe compilar en un entorno Codespace.

En caso de no cumplir con los puntos anteriores, el taller será evaluado con nota 1.0

Rúbrica

A. Árbol de decisión

(25 pts)

Criterio	Subcriterio	Pts	Descripción
Árbol de decisión	Implementación	8	El árbol de decisión se implementa correctamente mediante nodos y punteros.
Árbol de decisión	Evaluación correcta	8	El árbol evalúa correctamente los atributos del paciente y determina el servicio correspondiente.
Árbol de decisión	Recorridos	4	Se implementan correctamente los recorridos sobre el árbol de decisión.
Árbol de decisión	Diseño y justificación	5	El criterio de decisión está diseñado por el grupo y documentado en el README.

B. Árboles AVL

(40 pts)

Criterio	Subcriterio	Pts	Descripción
AVL	Inserción	8	Inserta correctamente pacientes en el AVL utilizando el ID como clave.
AVL	Búsqueda	6	Permite buscar correctamente un paciente por ID dentro del AVL.
AVL	Eliminación	6	Elimina correctamente un paciente por ID, manteniendo la estructura válida.
AVL	Altura / factor de balance	5	Calcula correctamente la altura del árbol y el factor de balance de cada nodo.
AVL	Rotaciones simples	6	Implementa correctamente las rotaciones simples izquierda y derecha.
AVL	Rotaciones dobles	6	Implementa correctamente las rotaciones dobles izquierda-derecha y derecha-izquierda.
AVL	Conservación del balance	3	El árbol se mantiene balanceado después de cada inserción.

C. Integración**(20 pts)**

Criterio	Subcriterio	Pts	Descripción
Carga de datos	Lectura del archivo	3	El archivo de pacientes se carga correctamente.
Queue	Queue y FIFO	4	Los pacientes se incorporan y procesan respetando el orden FIFO de la Queue.
Procesamiento	Envío al AVL correspondiente	5	El paciente es enviado correctamente al AVL del servicio determinado por el árbol de decisión.
Consultas	Consultas y recorridos	4	El sistema permite consultar el estado de los servicios y sus recorridos correctamente.
Robustez	Errores y casos borde	4	Se manejan correctamente archivos inexistentes, datos inválidos y pacientes duplicados.

D. Calidad y documentación**(15 pts)**

Criterio	Subcriterio	Pts	Descripción
Modularidad	Organización del código	4	El código está correctamente organizado y estructurado.
Documentación	README y documentación	4	El README documenta el criterio del árbol de decisión y el funcionamiento general del sistema.
Git	Control de versiones	3	El repositorio presenta un historial de commits coherente en GitHub.
Memoria	Liberación de memoria	4	El programa libera correctamente la memoria dinámica utilizada por los árboles.

Puntaje total: 100